📡 IOT – INDICE
1 Introduzione all'IoT
2 Hardware degli Smart Objects
3 Protocollo UDP
4 Protocollo TCP
5 IPv6
6 IEEE 802.15.4
7 MAC Energy-Efficient
8 6LoWPAN
9 RPL
10 CoAP
11 OS per IoT
12 Sicurezza IoT
13 LoRa e LoRaWAN

Guida Discorsiva all'Internet of Things

Questo documento presenta in modo narrativo e approfondito tutti gli argomenti del corso IoT del Prof. Gianluigi Ferrari (Università di Parma). L'obiettivo è capire non solo *cosa* fanno i protocolli e le tecnologie, ma soprattutto *perché* esistono, quali problemi risolvono e come si incastrano tra loro in un sistema IoT reale.

1. Intro IoT · 2. Hardware · 3. UDP · 4. TCP · 5. IPv6 · 6. 802.15.4 · 7. MAC · 8. 6LoWPAN · 9. RPL · 10. CoAP · 11. OS · 12. Security · 13. LoRa

🌐 1. Introduzione all'Internet of Things

Cos'è l'IoT e perché è nato

L'Internet of Things — letteralmente "Internet delle Cose" — rappresenta una delle evoluzioni più profonde nella storia delle reti digitali. Se l'Internet tradizionale collegava persone a persone, o persone a risorse informative come siti web, l'IoT ha come obiettivo quello di estendere la connettività a oggetti fisici del mondo reale: sensori ambientali, elettrodomestici, veicoli, macchinari industriali, dispositivi medici e molto altro. In altre parole, si tratta di dare una "voce digitale" a tutto ciò che ci circonda, facendolo diventare parte attiva della rete.

Il termine ha origini relativamente recenti ma si appoggia su una visione che risale alla fine degli anni Novanta. Nel 1997, l'ITU (International Telecommunication Union) pubblica il rapporto "The Internet of Things", che per la prima volta inquadra la sfida tecnologica di portare oggetti fisici — inizialmente tramite tecnologia RFID — sulla rete Internet. Nel 1999 viene fondato il centro Auto-ID al MIT, che sviluppa le prime tecnologie di identificazione automatica per la supply chain. Solo nel 2008 si tiene la prima conferenza internazionale dedicata all'IoT (IoT 2008, Zurigo), segnando la maturazione accademica e industriale del settore.

> 🔑 **DEFINIZIONE FONDAMENTALE**
> Le tecnologie e le soluzioni che permettono l'integrazione di dati e servizi del mondo reale nelle attuali reti informatiche sono spesso descritte con il termine ombrello di **Internet of Things**. L'IoT non è solo la connessione di dispositivi, ma la creazione di un ecosistema in cui oggetti fisici acquisiscono identità digitale e capacità di interazione autonoma.

Dal M2M all'IoT: una distinzione importante

Prima di parlare di IoT nel senso moderno, vale la pena capire il concetto di **Machine-to-Machine (M2M)**, che ne è il precursore diretto. La telemetria — ovvero la trasmissione remota di misure — è esistita per decenni: si pensi alle stazioni meteorologiche remote che inviano dati di temperatura e umidità via satellite, oppure ai pacemaker che trasmettono informazioni cardiache tramite apposite bobine di telemetria a bassa potenza. L'M2M generalizza questo concetto: è la comunicazione autonoma tra macchine, senza intervento umano, verso un centro di raccolta o elaborazione dati.

L'IoT rappresenta un salto qualitativo rispetto all'M2M: mentre l'M2M tende ad essere verticale (ogni industria sviluppa la propria soluzione proprietaria, con hardware, software e API dedicati), l'IoT punta all'interoperabilità tramite protocolli standard aperti, in particolare quelli della famiglia IP. Questo è cruciale: un mondo M2M frammentato in silos verticali impedisce economie di scala, rende difficile l'integrazione tra sistemi e aumenta enormemente i costi di sviluppo e manutenzione.

> 📌 **FRAMMENTAZIONE VERTICALE DELL'M2M**
> Il problema principale dell'M2M tradizionale è la cosiddetta *vertical fragmentation*: ogni settore (sicurezza, trasporti, sanità, utilities, manifattura, supply chain) sviluppa le proprie soluzioni proprietarie, reinventando ogni volta la ruota. L'IoT, attraverso la standardizzazione basata su IP, punta a fornire uno *Service Capability Layer* comune che tutte le applicazioni possano condividere tramite API aperte.

La scala dell'IoT: un cambiamento quantitativo e qualitativo

Uno degli aspetti più impressionanti dell'IoT è la scala. Secondo le stime di Ericsson e altri analisti, entro il 2020 si prevedevano circa 50 miliardi di oggetti connessi alla rete globale — un numero che supera di gran lunga il numero di esseri umani sul pianeta. Questa crescita non è semplicemente quantitativa: porta con sé sfide architetturali e tecnologiche che l'Internet tradizionale non ha mai dovuto affrontare.

Per comprendere la portata della sfida, basti considerare un singolo esempio fornito nelle slide del corso: una rete di smart object che misura la temperatura all'interno di un edificio, campionando due volte al minuto, produce **737.280 letture per anno per nodo**. Se il sistema è progettato per funzionare per dieci anni, si parla di oltre sette milioni di letture per singolo nodo. Moltiplicato per migliaia di nodi, si capisce perché il design di un routing protocol per LLN (Low-power and Lossy Networks) debba essere radicalmente diverso da quelli progettati per l'Internet tradizionale.

Sfide node-level e network-level

Il Prof. Ferrari distingue due categorie principali di sfide nell'IoT. Le **sfide a livello di nodo** riguardano i vincoli fisici e computazionali del singolo dispositivo: consumo energetico e dimensioni fisiche. Le **sfide a livello di rete** emergono dall'ensemble di tutti i nodi e riguardano la gestione del routing, l'indirizzamento, l'inaffidabilità dei link e la scala.

Queste due categorie sono profondamente interconnesse. Il consumo energetico di un nodo è determinato in larga parte dal suo comportamento di comunicazione: un protocollo di routing mal progettato che obbliga i nodi a stare costantemente in ascolto può decimare la vita della batteria. Viceversa, un protocollo di routing che tiene conto degli stati energetici dei nodi (come RPL fa con il Node Energy Object) può prolungare significativamente la vita dell'intera rete.

> ℹ️ **NODE-LEVEL CHALLENGES**
> **Consumo energetico:** La batteria è spesso l'unica fonte di energia disponibile. Il hardware designer minimizza le correnti di leakage e progetta modalità di sleep efficienti. Il software designer sfrutta al massimo le funzionalità hardware. I protocolli di rete devono nativamente supportare low-power operation. Vale la regola d'oro: *idle listening costa tanto quanto trasmettere.*
>
> **Dimensioni fisiche e costo:** Microprocessori più piccoli implicano meno velocità computazionale e meno memoria. Ogni linea di codice di un protocollo deve essere giustificata. La memoria tipica di uno smart object è dell'ordine di pochi kByte di RAM e poche decine di kByte di Flash.

Il ruolo della standardizzazione: IETF, IEEE e gli altri SDO

Un aspetto che il corso sottolinea con forza è l'importanza della standardizzazione aperta. In un ecosistema IoT, dispositivi di produttori diversi devono interoperare a tutti i livelli dello stack: fisico, di link, di rete, di trasporto e applicativo. Senza standard condivisi, ogni produttore crea una soluzione incompatibile con quella degli altri, annullando i benefici economici di scala.

L'**IETF** (Internet Engineering Task Force) ha svolto un ruolo fondamentale attraverso tre Working Group particolarmente rilevanti per l'IoT: il WG **6LoWPAN** (che ha sviluppato l'adaptation layer per IPv6 su IEEE 802.15.4), il WG **ROLL** (che ha prodotto RPL, il protocollo di routing per LLN) e il WG **CoRE** (che ha definito CoAP, il protocollo applicativo RESTful per dispositivi vincolati). Parallelamente, l'**IEEE** ha standardizzato le tecnologie di accesso al mezzo (PHY e MAC) tramite IEEE 802.15.4 per le WPAN a bassa potenza e IEEE 802.11 per le WLAN.

> 📖 **APPROFONDIMENTO (LIBRO)**
> Il libro "Interconnecting Smart Objects with IP" di Vasseur e Dunkels (Morgan Kaufmann, 2010) è il testo di riferimento principale del corso. Il Capitolo 1 fornisce un'eccellente panoramica sulla storia e le motivazioni dell'IoT, mentre i capitoli successivi approfondiscono ogni singolo strato dello stack protocollare. Si consiglia di leggere le introduzioni di ogni capitolo per capire il "perché" prima di approfondire il "come".

⚙️ 2. Hardware degli Smart Objects

I quattro componenti fondamentali

Ogni smart object, indipendentemente dalla sua applicazione specifica, è composto da quattro componenti hardware fondamentali. È utile ricordarseli con un'analogia con il corpo umano: la **radio/transceiver** funge da bocca e orecchie (comunicazione), il **microcontrollore** è il cervello (elaborazione), i **sensori e attuatori** sono le mani (interazione con il mondo fisico), e la **fonte di energia** è il cibo (alimentazione). Questa struttura elementare, combinata con software opportuno, è sufficiente per costruire sistemi IoT di grande complessità.

Il transceiver radio: la componente più energivora

Il transceiver radio è la componente più critica dal punto di vista del consumo energetico: consuma tipicamente **circa 10 volte più energia del microcontrollore o dei sensori**. Questo vale sia in trasmissione (modulazione del segnale) sia in ricezione (demodulazione), ma — e questo è il punto cruciale — vale anche in **idle listening**, ovvero quando il transceiver è acceso e in ascolto ma non sta effettivamente ricevendo nessun pacchetto utile. Questa osservazione è alla base di tutte le tecniche MAC per il risparmio energetico che vedremo più avanti.

I transceiver moderni variano enormemente in termini di capacità: da quelli semplici che gestiscono solo bit individuali, a quelli più sofisticati (SoC — System-on-Chip) che pacchettizzano i dati, costruiscono header e persino cifrano il payload usando metodi come AES-128. In un'ottica IoT, la scelta del transceiver influenza tutta l'architettura del sistema: un SoC più potente costa di più ma semplifica il software; un transceiver minimale richiede più elaborazione al microcontrollore ma riduce la dimensione e il costo del chip.

Il microcontrollore: il cervello vincolato

Il microcontrollore connette la radio con i sensori/attuatori ed esegue il software dello smart object. A differenza dei processori dei PC, i microcontrollori usati nell'IoT hanno dimensioni e prestazioni radicalmente ridotte: tipicamente pochi kByte di RAM e qualche decina-centinaia di kByte di Flash, con clock nell'ordine dei MHz (contro i GHz dei PC moderni). La memoria è divisa tra ROM (che contiene il comportamento permanente, ovvero il codice compilato) e RAM (che contiene i dati temporanei durante l'esecuzione).

Un punto fondamentale: i microcontrollori usati nell'IoT (con costi tipicamente inferiori a 10 dollari) hanno memoria sufficiente per stack protocollari ridotti ma non per implementazioni complete di tutto lo stack TCP/IP tradizionale. Questa è la ragione per cui esistono implementazioni compatte come **uIP** (micro-IP, solo 5 kB per TCP/IP completo) e **uIPv6** (11 kB), sviluppate nel contesto di Contiki OS.

Sensori, attuatori e fonti di energia

I sensori captano informazioni dal mondo fisico (temperatura, umidità, accelerazione, luce, pressione, segnali EMG, ecc.) e li convertono in segnali elettrici che il microcontrollore può leggere. Gli attuatori fanno il contrario: convertono segnali elettrici in azioni fisiche (apertura di valvole, attivazione di motori, emissione di suoni). La gamma va da sensori semplicissimi (un termistore) a sensori molto complessi (telecamere, accelerometri 9-DOF per IMU).

Per quanto riguarda l'alimentazione, la fonte più comune è una batteria al litio non ricaricabile, che con un'elettronica a basso consumo e software appropriato può garantire **anni di vita** anche con una singola batteria AA standard. Un'alternativa emergente è il **power scavenging** (energy harvesting): raccogliere energia dall'ambiente tramite celle solari, piezoelettricità (vibrazioni meccaniche) o induzione elettromagnetica (come negli RFID passivi). Quest'ultima tecnica è particolarmente interessante perché elimina completamente la necessità di batterie.

Lo spettro delle classi di microcontrollori: ARM Cortex

Un quadro molto utile per capire la varietà di dispositivi IoT è fornito dallo spettro dei microcontrollori basati su architettura ARM Cortex. Questa architettura è la più diffusa nell'IoT grazie alla sua efficienza energetica, alla disponibilità di tool di sviluppo e alla scalabilità. Il prof. Ferrari presenta una tabella di riferimento che vale la pena comprendere in profondità:

Classe MCU	Core ARM	RAM tipica	Flash tipica	Costo (IC)	OS / Stack	Applicazioni tipiche
Ultra low	Cortex-M0+	<4 kB	—	<1$	Nessuno	Sensore temperatura single-use
Low	M0, low M3	10 kB	64 kB	~2$	Embedded OS / nessuno	Sensore luce, motion sensor
Med	M3, M4	48 kB	256 kB	~5$	RTOS / Embedded OS	Lampadina smart, motion sensor avanzato
Med-High	M4	1 MB	1 MB	~8$	Embedded OS completo	Smartwatch, controller lavatrice
High	Cortex-A	>16 MB	>64 MB	~12$	Linux, Android	Gateway IoT, hub locale, smart pen

La tabella mostra chiaramente che non esiste un'unica risposta alla domanda "quale microcontrollore usare per IoT": dipende dal caso d'uso. Un sensore di temperatura che trasmette raramente e ha requisiti minimi può usare un Cortex-M0+ senza nessun OS; un gateway IoT che gestisce connessioni multiple e interfacce verso Internet richiede un Cortex-A con Linux.

> 🔑 **CONCETTO CHIAVE: ARM BUSINESS MODEL**
> ARM (ora Arm Holdings) non produce chip direttamente: vende *licenze* dell'architettura dei core a produttori di semiconduttori (STMicroelectronics, NXP, Nordic, Texas Instruments, ecc.) che costruiscono i propri SoC. Questo modello ha permesso una standardizzazione de-facto dell'architettura IoT, con l'ecosistema software (tool, librerie, OS) condiviso tra centinaia di chip diversi.

I sistemi operativi per smart objects

Anche i sistemi operativi per smart objects sono radicalmente diversi da quelli per PC o smartphone. Non c'è interfaccia grafica, la memoria è minima, e i pattern di esecuzione sono profondamente diversi. Esistono tre OS principali che il corso analizza in dettaglio (e che approfondiremo nella sezione 11):

Contiki (2003, SICS-Thingsquare) è stato il primo OS per smart objects a garantire la comunicazione IP tramite lo stack uIP (5 kB per TCP/IP, 11 kB per uIPv6). Usa il modello di programmazione dei *protothreads* — una forma di cooperative multitasking molto efficiente in termini di memoria. Include il simulatore **Cooja**, indispensabile per testare reti di centinaia di nodi virtuali prima del deployment reale.

TinyOS (2000, Berkeley-Stanford) nasce con un focus sulle wireless sensor network e sull'ambito accademico. È scritto in *nesC*, un dialetto del C ottimizzato per il modello event-driven. I programmi TinyOS reagiscono a eventi (ricezione di un pacchetto, scadenza di un timer) piuttosto che eseguire loop continui, il che li rende naturalmente efficienti ma più complessi da scrivere.

FreeRTOS è progettato per sistemi embedded con garanzie real-time, tipicamente applicazioni di controllo industriale. Usa un modello di programmazione preemptive e multithreaded, che richiede più memoria ma offre maggiore flessibilità. È portato su oltre 50 microcontrollori/controllori ed è ampiamente usato nell'industria.

📦 3. Il Protocollo UDP

UDP nel contesto del TCP/IP stack

Prima di parlare di UDP specificamente, vale la pena ricordare il contesto architetturale in cui si colloca. Il modello TCP/IP suddivide la comunicazione di rete in livelli, ciascuno con responsabilità ben definite. Il **livello di rete** (IP) si occupa di instradare i pacchetti da sorgente a destinazione attraverso la rete, identificando i dispositivi tramite indirizzi IP a 32 o 128 bit. Il **livello di trasporto** (TCP o UDP) fornisce comunicazione end-to-end tra *processi applicativi* che girano su dispositivi diversi — e qui sta il punto cruciale: mentre IP identifica un dispositivo, i protocolli di trasporto identificano un'applicazione specifica su quel dispositivo tramite i *numeri di porta*.

UDP è il protocollo di trasporto più semplice possibile: si può pensare a UDP come a IP con l'aggiunta di due sole funzionalità — il **multiplexing/demultiplexing** tramite numeri di porta (per distinguere tra le varie applicazioni sullo stesso host) e un **checksum opzionale** per la rilevazione degli errori sul payload. Nient'altro. Non c'è connessione, non c'è garanzia di consegna, non c'è ordinamento, non c'è controllo di flusso.

La struttura del datagram UDP

L'header UDP è sorprendentemente compatto: soli **8 byte totali**, composti da quattro campi da 2 byte ciascuno. Il *Source Port* identifica il processo mittente (da 0 a 65.535, ovvero 2^{16}−1); il *Destination Port* identifica il processo destinatario; il *Length* specifica la lunghezza totale del datagram UDP (header + dati), con un minimo di 8 byte (datagram vuoto) e un massimo di 65.535 byte; infine il *Checksum* copre l'header UDP e parte dell'header IP (il cosiddetto pseudo-header). In IPv4 il checksum è opzionale; in IPv6 è obbligatorio.

Source Port (16 bit)	Destination Port (16 bit)	Length (16 bit)	Checksum (16 bit)

Header UDP: 8 byte totali (4 campi × 2 byte)

Perché qualcuno dovrebbe usare UDP invece di TCP?

A prima vista, UDP sembra il "protocollo scadente": non garantisce la consegna, non riordina i pacchetti, non recupera le perdite. Ma questa semplicità è esattamente il suo punto di forza in molti scenari. Le motivazioni principali per scegliere UDP sono quattro, e tutte particolarmente rilevanti per l'IoT:

1. Controllo fine-grained su quando inviare: Con UDP, non appena un'applicazione scrive nel socket, UDP impacchetta i dati e invia subito. Con TCP invece si deve attendere la negoziazione della connessione, il controllo di congestione e altri meccanismi che introducono latenza. Per applicazioni IoT che devono inviare singole misure in modo rapido e sporadico, questa latenza è inaccettabile.

2. Nessun overhead di connessione: TCP richiede il *three-way handshake* prima di poter inviare il primo byte di dati (3 round-trip time!). UDP "spara" subito. In una rete IoT con link lenti (ad esempio 250 kb/s su IEEE 802.15.4), questi round-trip possono essere significativi, e il costo energetico di mantenere una connessione aperta è rilevante.

3. Nessuno stato di connessione: TCP mantiene buffer, numeri di sequenza, timer e altri parametri per ogni connessione attiva. UDP è completamente stateless: nessuna allocazione di risorse, nessun timeout da gestire, nessuna complessità. Per un server che deve gestire migliaia di client (ad es. un gateway IoT con molti sensori), questa assenza di stato è un vantaggio enorme in termini di scalabilità e memoria.

4. Header piccolo: 8 byte contro i 20 byte minimi di TCP (spesso di più con opzioni). Su un frame IEEE 802.15.4 da 127 byte massimi, ogni byte risparmiato nell'header è un byte in più per il payload applicativo. UDP usa il 6,3% del frame; TCP usa il 15,7%, e spesso di più.

> ⚠️ **UDP PER IOT: VANTAGGI E SVANTAGGI**
> **Vantaggi IoT:** Bassissimo overhead (header 8B), semplicità adatta a molte applicazioni (monitoraggio temperatura: perdere una lettura non è critico), buona compatibilità con multicast sottostante.
>
> **Svantaggi IoT:** Nessun meccanismo di recovery dalla perdita di pacchetti → l'applicazione deve gestire essa stessa la ritrasmissione se necessario. Le reti smart object hanno frame piccoli e UDP non fornisce meccanismi di frammentazione → l'applicazione deve scegliere dimensioni appropriate dei pacchetti.

Applicazioni che usano UDP e rilevanza nell'IoT

UDP è la scelta naturale per applicazioni di **streaming multimediale** (audio/video conferencing, gaming online) dove un pacchetto perso è preferibile a un ritardo causato dalla ritrasmissione. Per una videochiamata, un frame corrotto viene semplicemente ignorato; un frame ritardato di 300ms è invece inaccettabile. Allo stesso modo, per protocolli di query semplici come **DNS** (Domain Name System), la semplicità di UDP si adatta perfettamente: si invia la query e si attende la risposta; se arriva un errore o il timeout scade, si ritrasmette a livello applicativo.

Nel contesto IoT, UDP è il protocollo di trasporto su cui è costruito **CoAP** (Constrained Application Protocol), il protocollo applicativo standard per IoT. Questo non è un caso: CoAP implementa su di sé i meccanismi di affidabilità opzionale (messaggi CON con ACK) che servono, ma solo quando l'applicazione li richiede esplicitamente, evitando l'overhead di TCP nelle situazioni in cui non sono necessari.

🔗 4. Il Protocollo TCP

La filosofia di TCP: affidabilità su un canale inaffidabile

TCP (Transmission Control Protocol) è il protocollo di trasporto orientato alla connessione e affidabile di Internet. Nasce dalla necessità di fornire un servizio di *byte stream* affidabile su un livello di rete (IP) che offre solo un servizio *best-effort* — ovvero IP fa del suo meglio per consegnare i pacchetti, ma non garantisce né la consegna né l'ordine. TCP si costruisce sopra questo servizio inaffidabile e fornisce tre garanzie fondamentali: **rilevazione e correzione degli errori**, **riordinamento dei segmenti** e **controllo di flusso**.

Per realizzare queste garanzie, TCP usa tre meccanismi chiave che è importante capire in profondità: gli **acknowledgment e le ritrasmissioni**, i **numeri di sequenza** e la **sliding window**.

Il Three-Way Handshake: come nasce una connessione TCP

Prima che i dati possano fluire, TCP richiede l'apertura esplicita di una connessione. Questo avviene tramite il famoso **three-way handshake**, una sequenza di tre messaggi che permette ai due host di sincronizzarsi sui numeri di sequenza iniziali (ISN — Initial Sequence Numbers) e di confermare che entrambi sono pronti a comunicare.

L'host A invia un segmento **SYN** (synchronize) con il proprio ISN, diciamo X. L'host B risponde con un segmento **SYN-ACK**, che contiene sia il proprio ISN (diciamo Y) sia un ACK per l'ISN di A (X+1, ovvero "ho ricevuto il byte X, mi aspetto il byte X+1"). Infine, A invia un **ACK** per l'ISN di B (Y+1), completando la sincronizzazione. Dopo questo scambio, entrambi gli host conoscono i numeri di sequenza dell'altro e la connessione è pronta per il trasferimento dati.

> ℹ️ **PERCHÉ L'ISN NON PARTE DA ZERO?**
> Il numero di sequenza iniziale non viene fissato a zero per motivi pratici: gli indirizzi IP e i numeri di porta vengono riutilizzati nel tempo, e un vecchio pacchetto ancora in transito nella rete potrebbe essere erroneamente associato a una nuova connessione. Per evitare questo problema, l'ISN viene derivato da un orologio a 32 bit che scorre ogni 4 microsecondi, garantendo che lo stesso ISN non venga riutilizzato per almeno 4,55 ore.

Numeri di sequenza e acknowledgment

Ogni byte trasmesso da TCP ha un numero di sequenza univoco. Il *sequence number* nel header TCP indica il numero del primo byte nel segmento corrente; l'*acknowledgment number* indica il numero del prossimo byte atteso dal ricevente. Questa semantica di *cumulative acknowledgment* significa che un ACK con valore N conferma la ricezione di tutti i byte da 0 a N-1.

L'header TCP è composto da 20 byte minimi e include: i numeri di porta sorgente e destinazione (4 byte), il numero di sequenza (4 byte), il numero di acknowledgment (4 byte), la lunghezza dell'header e i flag (2 byte — dove i flag includono SYN, FIN, RST, PSH, URG, ACK), la finestra pubblicizzata (2 byte), il checksum (2 byte) e il urgent pointer (2 byte).

Il problema del timeout: stimare l'RTT

TCP deve scegliere quanto tempo attendere prima di considerare un pacchetto perso e ritrasmettere. Questo timeout è critico: se è troppo breve, si ritrasmette inutilmente consumando banda e energia; se è troppo lungo, si perdono secondi preziosi ogni volta che un pacchetto va perso. La soluzione di TCP è **stimare dinamicamente l'RTT** (Round-Trip Time) osservando i tempi di risposta degli ACK.

TCP mantiene uno *Smoothed RTT* usando una media esponenzialmente ponderata:

```
EstimatedRTT = α × EstimatedRTT + (1 − α) × SampleRTT
```

dove α tipicamente vale 0.875 (ovvero si dà peso 7/8 alla stima corrente e 1/8 alla nuova misura). Il timeout viene poi fissato a `Timeout = 2 × EstimatedRTT` per aggiungere un margine di sicurezza. Questa stima adattiva è fondamentale: un nodo IoT su una rete wireless può avere RTT molto variabile a causa delle perdite, e un timeout fisso porterebbe a comportamenti sub-ottimali.

La Sliding Window: perché stop-and-wait è inefficiente

Il meccanismo più semplice di controllo del flusso è il **stop-and-wait**: il mittente invia un segmento, poi si ferma e attende l'ACK prima di inviare il successivo. Questo funziona ma è terribilmente inefficiente: in un link da 1,5 Mbps con RTT di 45 ms, il prodotto banda-ritardo è 67,5 kbit = 8 kByte. Con segmenti da 1 kB e stop-and-wait, si utilizza solo 8 kbit / 45 ms ≈ 182 kbps su 1,5 Mbps disponibili — appena il 12% della capacità del link!

La soluzione è la **sliding window**: si consente al mittente di avere più segmenti "in volo" contemporaneamente, senza aspettare l'ACK di ognuno. Il ricevente pubblica una *advertised window* nell'header TCP, indicando quanti byte è in grado di ricevere e bufferizzare in questo momento. Il mittente non può inviare più dati di quanti la finestra consenta. Man mano che il ricevente svuota il buffer (consegnando i dati all'applicazione), può pubblicare una finestra più grande, permettendo al mittente di inviare altri dati.

Fast Retransmit: ritrasmissione rapida senza aspettare il timeout

Il timeout è un meccanismo di sicurezza, ma può introdurre latenze significative: se un pacchetto va perso, il mittente aspetta che scada il timeout (che può essere di diversi secondi) prima di ritrasmettere. La **fast retransmission** è un'ottimizzazione che sfrutta il meccanismo di sliding window per rilevare le perdite più rapidamente.

Ecco il ragionamento: se il pacchetto N è perso ma i pacchetti N+1, N+2, N+3 arrivano correttamente, il ricevente continua ad inviare ACK per N (il "cumulative ACK" può solo confermare i byte in ordine). Il mittente riceve quindi **tre ACK duplicati** per N — che è un segnale forte che N è andato perso. Non serve aspettare il timeout: il mittente ritrasmette N immediatamente. Questa tecnica si chiama *triple duplicate ACK fast retransmit* ed è molto più efficace quando c'è una perdita isolata in un flusso continuo.

> ⚠️ **TCP PER IOT: VANTAGGI E SVANTAGGI**
> **Vantaggi:** Le reti IoT sono tipicamente lossy (link wireless, interferenze) → il trasporto affidabile di TCP è utile. TCP può essere compresso (header compression in 6LoWPAN). L'opzione MSS (Maximum Segment Size) permette di adattare la dimensione dei segmenti al frame piccolo di 802.15.4.
>
> **Svantaggi:** Le implementazioni ridotte di TCP per smart objects non implementano la sliding window completa (solo un pacchetto attivo alla volta) né gli ACK ritardati. Il three-way handshake introduce 3 RTT di latenza prima di poter trasmettere dati.

🌍 5. IPv6

Perché IPv4 non basta: il problema dell'esaurimento degli indirizzi

IPv4 (RFC 791, 1981) funziona sostanzialmente invariato da oltre quarant'anni, ed è un tributo alla brillantezza dei suoi progettisti. Tuttavia, la sua limitazione fondamentale — gli indirizzi a 32 bit — ha creato problemi crescenti con l'esplosione di Internet e, in misura ancora maggiore, con l'avvento dell'IoT. Con 2^{32} ≈ 4,3 miliardi di indirizzi teorici (molti meno in pratica, data la struttura gerarchica), si è iniziato presto a correre ai ripari con tecniche come il CIDR (Classless Inter-Domain Routing) e soprattutto il NAT (Network Address Translation).

Il NAT permette a molti dispositivi con indirizzi privati di condividere un unico indirizzo pubblico, risolvendo il problema dell'esaurimento nel breve termine ma creando nuovi problemi: introduce stato nella rete (violando il principio end-to-end di Internet), non funziona con alcuni protocolli applicativi che incorporano indirizzi IP nel payload (FTP, SIP, DNS), ed è incompatibile con IPSec in alcune configurazioni. Per l'IoT, il NAT è particolarmente problematico: un dispositivo IoT dietro NAT non è direttamente raggiungibile dall'esterno, il che complica enormemente le architetture di comunicazione.

La soluzione definitiva è **IPv6** (RFC 2460), che porta gli indirizzi a 128 bit, fornendo 2^{128} ≈ 3,4 × 10^{38} indirizzi unici — un numero sufficientemente grande da assegnare un indirizzo globalmente unico a ogni granello di sabbia sul pianeta, senza bisogno di NAT.

L'header IPv6: più semplice ma più grande

L'header base IPv6 è di **40 byte fissi** — il doppio dei 20 byte minimi di IPv4 — ma è strutturato in modo più pulito e efficiente. Contiene solo 8 campi (contro i 13 di IPv4), ed è allineato a 64 bit per ottimizzare l'elaborazione sui processori moderni. I campi rimossi rispetto a IPv4 includono il checksum dell'header (eliminato perché TCP e UDP calcolano già il proprio checksum, incluso uno pseudo-header), il campo di frammentazione (in IPv6 la frammentazione avviene solo alla sorgente, non nei router intermedi) e le opzioni variabili (spostate negli Extension Headers).

I campi dell'header base IPv6 sono: *Version* (4 bit, sempre "6"), *Traffic Class* (8 bit, per la qualità del servizio, analogo al DSCP di IPv4), *Flow Label* (20 bit, per identificare flussi che richiedono gestione speciale), *Payload Length* (16 bit, lunghezza del payload escludendo l'header base), *Next Header* (8 bit, identifica il header successivo — extension header o protocollo di trasporto), *Hop Limit* (8 bit, analogo al TTL di IPv4), e i due indirizzi da 128 bit ciascuno.

Extension Headers: la flessibilità di IPv6

Una delle idee più eleganti di IPv6 è il sistema di **Extension Headers**: invece di gonfiare l'header base con opzioni raramente usate, IPv6 permette di concatenare header opzionali tra l'header base e il payload. Ogni extension header è identificato dal campo *Next Header* dell'header precedente, creando una catena che termina con il protocollo di trasporto (TCP=6, UDP=17) o con "no next header" (59).

Gli extension header standard includono: *Hop-by-Hop Options* (tipo 0, processato da ogni router lungo il percorso — include l'opzione Jumbo Payload per frame >65.535 byte), *Routing Header* (tipo 43, specifica una lista di router da attraversare), *Fragment Header* (tipo 44, usato per la frammentazione eseguita dalla sorgente), *Authentication Header* (tipo 51, per IPSec AH), *ESP Header* (tipo 50, per IPSec ESP), e *Destination Options* (tipo 60, processato solo dalla destinazione). Questa struttura permette di aggiungere nuove funzionalità a IPv6 senza modificare l'header base.

Indirizzi IPv6: gerarchie, tipi e rappresentazione

Gli indirizzi IPv6 a 128 bit si rappresentano come 8 gruppi da 16 bit (4 cifre esadecimali ciascuno), separati da ":", per esempio `2000:0000:0000:0040:0100:A000:0000:1234`. Alcune regole di semplificazione permettono di omettere gli zeri iniziali in ogni gruppo e di sostituire una sequenza di gruppi tutti zero con "::" (che può apparire una sola volta per indirizzo).

IPv6 distingue tre tipi di indirizzi: **unicast** (uno-a-uno), **anycast** (uno al più vicino, allocati dallo spazio unicast) e **multicast** (uno-a-molti, con prefisso FF00::/8). Non esiste il broadcast in IPv6, sostituito da specifici indirizzi multicast. Gli indirizzi unicast si dividono a loro volta in *global unicast* (instradabili su Internet, prefisso 2000::/3), *link-local* (validi solo sul link locale, prefisso FE80::/10, obbligatori per ogni interfaccia) e *site-local* (deprecated).

> 🔑 **LINK-LOCAL E SLAAC: AUTOCONFIGURAZIONE STATELESS**
> Ogni interfaccia IPv6 si assegna automaticamente un indirizzo link-local con prefisso FE80::/10. L'Interface ID (64 bit bassi) viene derivato dall'indirizzo MAC a 64 bit (EUI-64) o da 48 bit modificato. Questa capacità di auto-configurazione — SLAAC (Stateless Address Autoconfiguration) — è fondamentale per l'IoT: i dispositivi si connettono e si configurano da soli, senza bisogno di un server DHCP.

ICMPv6 e Neighbor Discovery Protocol

ICMPv6 non è semplicemente la versione per IPv6 dell'ICMP di IPv4: è un protocollo molto più ricco che incorpora le funzionalità di ARP (Address Resolution Protocol) e IGMP (Internet Group Management Protocol) di IPv4, più molte funzionalità nuove. ICMPv6 è essenziale per il funzionamento di IPv6 e viene usato per: segnalazione di errori (Destination Unreachable, Packet Too Big, Time Exceeded), diagnostica (Echo Request/Reply, il "ping"), e il fondamentale **Neighbor Discovery Protocol (NDP)**.

NDP risolve una serie di problemi critici nella gestione di IPv6 su un link. Il **Neighbor Solicitation/Advertisement** sostituisce l'ARP di IPv4: quando un nodo deve inviare un pacchetto a un vicino ma non conosce il suo indirizzo MAC, invia un Neighbor Solicitation in multicast; il vicino risponde con un Neighbor Advertisement contenente il suo MAC. Il **Router Solicitation/Advertisement** permette agli host di scoprire i router sul link e ottenere il prefisso di rete, fondamentale per SLAAC. La **Duplicate Address Detection (DAD)** verifica che l'indirizzo che un nodo vuole usare non sia già usato da altri sul link.

Meccanismi di transizione da IPv4 a IPv6

Il passaggio da IPv4 a IPv6 non può avvenire dall'oggi al domani: richiede meccanismi di coesistenza. I tre approcci principali sono il **dual stack** (un dispositivo implementa entrambe le versioni di IP e usa quella appropriata in base al destinatario), il **tunneling** (i pacchetti IPv6 vengono incapsulati in pacchetti IPv4 per attraversare reti IPv4-only) e la **translation** (un gateway traduce tra IPv4 e IPv6, necessario quando dispositivi che supportano solo una versione devono comunicare con dispositivi che supportano solo l'altra).

> 📌 **IPV6 E IOT: UN BINOMIO ESSENZIALE**
> Per l'IoT, IPv6 non è un'opzione ma una necessità. Con decine di miliardi di dispositivi connessi, IPv4 semplicemente non ha abbastanza indirizzi. Ma IPv6 porta molto di più: SLAAC elimina la dipendenza da DHCP (una significativa infrastruttura DHCP), gli indirizzi link-local permettono la comunicazione locale senza router; la multicast consentono routing protocol efficienti come RPL. L'IETF ha sviluppato 6LoWPAN proprio per adattare IPv6 alle reti IoT a con i loro vincoli estremi.

📡 6. Standard IEEE 802.15.4

Perché 802.15.4 e non WiFi?

Una domanda legittima quando si studia l'IoT è: perché non usare semplicemente il WiFi (IEEE 802.11) per connettere gli smart object? Dopotutto è uno standard ampiamente diffuso, con miliardi di dispositivi compatibili. La risposta sta in tre parole: **consumo energetico, costo e dimensioni**. Un modulo WiFi consuma tipicamente 10-100 volte più energia di un modulo IEEE 802.15.4, ha chip 802.15.4 possono costare poche centesimi contro qualche euro del WiFi; e le dimensioni fisiche sono corrispondentemente diverse.

IEEE 802.15.4, sviluppato dal gruppo di lavoro 802.15 dell'IEEE con focus sulle Personal Area Network (PAN) a bassa potenza, è progettato proprio per dispositivi con severi vincoli energetici e di costo. Il suo profilo tecnico riflette queste priorità: **data rate massimo di 250 kb/s** (contro i Mbps o Gbps del WiFi), **potenza di trasmissione massima 1 mW**, **range nominale di poche decine di metri**, e crucialmente un **MTU (Maximum Transmission Unit) di soli 127 byte**. Quest'ultimo dato è forse il più impattante sull'intera architettura IoT: tutti i protocolli superiori devono adattarsi a questi 127 byte.

Il layer fisico (PHY): canali e modulazione

IEEE 802.15.4 definisce tre bande di frequenza operative. In **Europa** è disponibile un canale nella banda 868 MHz (canale 0); negli **Stati Uniti** sono disponibili 10 canali nella banda 915 MHz (canali 1-10); in tutto il **mondo** (inclusa l'Europa) sono disponibili 16 canali nella banda 2,4 GHz (canali 11-26). I canali nella banda 2,4 GHz sono i più usati in pratica, con velocità di 250 kb/s e modulazione **O-QPSK** (Offset-QPSK). I canali 0-10 a frequenze inferiori usano BPSK, con velocità inferiori (20-40 kb/s) ma migliore penetrazione degli ostacoli.

Un problema rilevante è l'interferenza di IEEE 802.11 (WiFi) nella banda 2,4 GHz. Poiché i canali dei due standard si sovrappongono parzialmente, un nodo 802.15.4 vicino a un punto di accesso WiFi attivo può sperimentare interferenze significative, aumentando la lossiness del link. Questo è uno dei motivi per cui i protocolli IoT devono essere progettati per reti *lossy by design*.

Il layer fisico fornisce anche il meccanismo di **CCA (Clear Channel Assessment)**, ovvero la valutazione se il canale è libero o occupato prima di trasmettere. CCA può operare in tre modalità: misura dell'energia del segnale (Energy Detection), demodulazione del segnale per verificare se è un frame 802.15.4, o la combinazione delle due.

Il layer MAC: accessi al canale e tipi di nodi

Il layer MAC di IEEE 802.15.4 usa **CSMA/CA** (Carrier Sense Multiple Access with Collision Avoidance) come meccanismo base di accesso al canale: prima di trasmettere, un nodo esegue CCA; se il canale è occupato, esegue un backoff casuale e riprova. Le collisioni non possono essere rilevate in wireless (il segnale trasmesso maschera quello ricevuto), quindi si usa la *collision avoidance* piuttosto che la collision detection di Ethernet.

Lo standard definisce due tipi di dispositivi: i **FFD (Full Function Devices)**, che implementano il MAC completo e possono fungere da PAN coordinator o router, e i **RFD (Reduced Function Devices)**, con funzionalità MAC ridotta (non possono essere PAN coordinator). La rete è identificata da un **PAN ID a 16 bit** e supporta tre topologie: *star*, *mesh* e *cluster tree*.

L'indirizzamento è su due livelli: ogni dispositivo ha un **indirizzo a 64 bit** globalmente unico (assegnato dal produttore, con 24 bit OUI dell'IEEE e 40 bit manufacturer-specific) e può ricevere un **indirizzo corto a 16 bit** assegnato dal PAN coordinator al momento dell'associazione alla rete. L'indirizzo lungo garantisce l'unicità globale; quello corto riduce l'overhead degli header sui frame limitati a 127 byte.

Il formato dei frame 802.15.4

Il frame fisico (PHY) è strutturato in tre parti: un **preambolo** (per la sincronizzazione del ricevitore), un **SFD** (Start of Frame Delimiter, che indica la fine del preambolo) e un campo **Length** (7 bit, da 0 a 127, che indica la lunghezza del PSDU — PHY Service Data Unit). Il PSDU contiene il frame MAC.

Il frame MAC comprende: un **Frame Control** (con flag per indicare il tipo di frame, la richiesta di ACK, la versione del frame, ecc.), un **Sequence Number** (per il matching con gli ACK), i campi di **indirizzamento** (variabili, con PAN ID sorgente/destinazione e indirizzi), il **payload** e un **FCS** (Frame Check Sequence, CRC-16 di tutto il frame). È importante notare che il payload effettivamente disponibile per i layer superiori dipende dall'overhead MAC: nel caso peggiore (indirizzi lunghi a 64 bit e header di sicurezza AES) rimangono solo 21-41 byte per i dati dell'applicazione.

> ℹ️ **IL BUDGET ENERGETICO DI 802.15.4**
> Un dato sorprendente: su IEEE 802.15.4, il consumo in ricezione è superiore a quello in trasmissione. Questo perché la ricezione richiede l'attivazione completa dei circuiti di demodulazione anche quando non arriva nessun segnale utile. Conseguenza diretta: mantenere la radio accesa in modalità "ascolto" consuma tanta energia quanto trasmettere, rendendo l'*idle listening* il nemico principale dell'efficienza energetica nelle reti IoT.

IEEE 802.11 e PLC: confronto per IoT

Oltre a 802.15.4, il corso analizza brevemente altri due standard di comunicazione rilevanti per l'IoT. IEEE 802.11 (WiFi) ha il vantaggio dell'ubiquità (miliardi di dispositivi, infrastruttura già presente) e data rate elevati, ma soffre di consumo energetico molto superiore a 802.15.4. Le nuove generazioni a bassa potenza di 802.11 stanno migliorando, ma il gap energetico rimane significativo per applicazioni battery-powered.

Le **Power Line Communications (PLC)** usano le linee elettriche esistenti come canale di trasmissione dati, eliminando la necessità di batterie (i dispositivi si alimentano direttamente dalla rete elettrica). Il payload dati viene sovrapposto al segnale a 50/60 Hz tramite modulazione. Il vantaggio principale è la disponibilità pervasiva della rete elettrica; lo svantaggio è il rumore elevato introdotto dai carichi elettrici connessi. Le PLC usano CSMA/CA e non richiedono routing (tutti i nodi sullo stesso segmento di rete elettrica sono sulla stessa LAN).

⚡ 7. Protocolli MAC Energy-Efficient

Il problema dell'idle listening: il nemico nascosto

Come abbiamo visto, il consumo energetico del transceiver radio è la principale causa di scarsa vita della batteria negli smart object. Ma qual è la fonte principale di questo consumo? Non è, come si potrebbe pensare, la trasmissione — dopotutto un nodo trasmette solo quando ha dati da mandare. La fonte principale è l'**idle listening**: mantenere il transceiver acceso in ricezione in attesa di possibili trasmissioni in arrivo, anche quando la rete è silenziosa.

Sperimentalmente si è dimostrato che il consumo in idle listening è comparabile a quello in ricezione attiva (e spesso superiore a quello in trasmissione). Un nodo che "ascolta" continuamente consuma la batteria quasi quanto uno che trasmette continuamente. La soluzione ovvia è spegnere il transceiver il più possibile — ma come fa il nodo a sapere quando è opportuno accendersi per ricevere dati destinati a lui?

Questo è il problema fondamentale che i protocolli MAC energy-efficient cercano di risolvere. Le altre fonti di consumo "sprecato" includono: le **collisioni** (due nodi trasmettono contemporaneamente, entrambi i pacchetti si perdono), l'**overhearing** (un nodo riceve ed elabora pacchetti destinati ad altri), e l'overhead dei messaggi di controllo.

B-MAC: la semplicità del Low Power Listening

B-MAC (Berkeley MAC, 2004) affronta il problema dell'idle listening con un'idea elegante nella sua semplicità: **Low Power Listening (LPL)**. Anziché tenere la radio sempre accesa, ogni nodo si sveglia periodicamente per breve tempo, esegue un CCA per rilevare attività sul canale, e se non c'è nulla torna a dormire. Se rileva un segnale, rimane sveglio per ricevere il pacchetto.

Il problema è: come fa il mittente a sincronizzarsi con il ciclo di sleep del ricevente, che non conosce? La soluzione di B-MAC è inviare un **preambolo molto lungo** — abbastanza lungo da coprire l'intero intervallo di sleep del ricevente. In questo modo, indipendentemente da quando il ricevente si sveglia durante il preambolo, lo troverà e rimarrà sveglio per ricevere il pacchetto successivo. La durata del preambolo deve essere almeno uguale all'intervallo di sleep del nodo.

B-MAC è robusto (non richiede sincronizzazione) e semplice da implementare, ma ha un difetto significativo: tutti i nodi nel range del mittente si svegliano e rimangono svegli per tutta la durata del preambolo, anche quelli a cui il pacchetto non è destinato (**overhearing**). In reti dense questo causa uno spreco energetico.

X-MAC: il preambolo strobed con l'indirizzo del destinatario

X-MAC (2006) risolve elegantemente il problema dell'overhearing di B-MAC con due modifiche principali. Prima: il preambolo non è lungo e continuo, ma è sostituito da una serie di **brevi pacchetti di preambolo strobed** (lampeggianti), tra cui il mittente inserisce brevi pause. Seconda: ogni pacchetto di preambolo contiene l'**indirizzo del destinatario**. In questo modo, i nodi che si svegliano e sentono il preambolo ma vedono che non sono il destinatario possono tornare immediatamente a dormire anziché aspettare l'intero ciclo.

Un ulteriore vantaggio: se il destinatario si sveglia durante il preambolo e lo riceve, può inviare un **early ACK** al mittente, che smette subito di trasmettere preamboli e invia direttamente il dato. Questo riduce la latenza nei casi in cui il destinatario si sveglia. X-MAC combina quindi l'approccio asincrono di B-MAC con un'ottimizzazione significativa per la densità di rete.

WiseMAC: imparare il ritmo del vicino

Un'altra idea interessante è quella di **WiseMAC**: anziché usare un preambolo lungo per coprire l'intero intervallo di sleep, il mittente *impara* il ciclo di sveglia del destinatario osservando i tempi degli ACK. Il destinatario codifica il proprio schema di sveglia nell'ACK (il campo "wake-up pattern"), e il mittente usa questa informazione per iniziare a trasmettere il preambolo appena *prima* che il destinatario si svegli. Questo riduce drasticamente la lunghezza del preambolo necessario e, di conseguenza, il consumo energetico del mittente.

S-MAC: il protocollo schedulato con listen/sleep periodico

S-MAC (Sensor MAC, 2002) adotta un approccio diverso: invece di essere reattivo (come B-MAC/X-MAC), è **schedulato**. Ogni nodo annuncia il proprio schedule di listen/sleep ai vicini, e i vicini cercano di sincronizzarsi su schedule compatibili. Un frame temporale è diviso in un periodo di listen (durante il quale i nodi sono svegli e possono comunicare) e un periodo di sleep (durante il quale tutti dormono). La comunicazione avviene solo nei periodi di listen, usando RTS/CTS per la collision avoidance.

S-MAC riduce l'idle listening perché i nodi dormono la maggior parte del tempo. Ma introduce un problema: se due nodi adiacenti hanno schedule diversi, devono rimanere svegli durante entrambi gli schedule per poter comunicare con entrambi i vicini, riducendo l'efficienza. Inoltre, lo schedule fisso non si adatta bene al traffico variabile.

ContikiMAC: il protocollo adattivo di Contiki

ContikiMAC (2011) è il protocollo MAC utilizzato di default nell'OS Contiki. Combina le idee di LPL di B-MAC con ottimizzazioni significative. Il suo meccanismo chiave è il **phase lock**: il mittente osserva quando il destinatario si sveglia (imparando la sua fase di duty cycling) e sincronizza le proprie trasmissioni per arrivare appena il destinatario si sveglia, riducendo il tempo di trasmissione del pacchetto necessario a quasi zero una volta che la fase è appresa.

ContikiMAC ottimizza anche il comportamento in broadcast: per i pacchetti broadcast, non potendo aspettare un ACK specifico, il mittente trasmette il pacchetto ripetutamente per un intero ciclo di sleep, assicurandosi che tutti i vicini lo ricevano indipendentemente dalla loro fase di sleep. I timestamp inseriti nelle trasmissioni permettono ai riceventi di calcolare la fase del mittente.

> 🔑 **RIASSUNTO PROTOCOLLI MAC ENERGY-EFFICIENT**
> **B-MAC:** LPL con preambolo lungo, semplice, problema di overhearing.
> **X-MAC:** Preambolo strobed con indirizzo destinatario, early ACK, risolve overhearing.
> **WiseMAC:** Impara lo schedule del vicino dall'ACK, preambolo adattivo.
> **S-MAC:** Schedule fisso listen/sleep, basso idle listening, latenza aumentata.
> **ContikiMAC:** Phase lock adattivo, efficiente, usato in Contiki OS.

🔌 8. 6LoWPAN

Il problema fondamentale: IPv6 su 802.15.4

A questo punto del corso, abbiamo due pezzi del puzzle: IPv6 (con i suoi header da 40 byte fissi) e IEEE 802.15.4 (con i suoi frame da 127 byte massimi). Il problema è evidente: IPv6 richiede un MTU minimo di 1280 byte, mentre 802.15.4 ha un MTU *massimo* di 127 byte. C'è uno scarto di quasi due ordini di grandezza. Senza un livello di adattamento, IPv6 su 802.15.4 semplicemente non funzionerebbe.

Il gruppo di lavoro IETF 6LoWPAN (avviato nel 2004) ha sviluppato l'**adaptation layer 6LoWPAN**, che si inserisce tra il layer IPv6 e il layer 802.15.4 e fornisce tre servizi fondamentali: **frammentazione e riassemblaggio** (per gestire l'MTU mismatch), **compressione degli header** (per ridurre l'overhead di IPv6 e UDP), e **supporto al routing multi-hop a livello link** (mesh-under forwarding).

Per capire quanto sia stringente il vincolo, consideriamo il caso peggiore: un frame 802.15.4 da 127 byte totali, con header MAC che include gli indirizzi lunghi (64 bit) sia per sorgente che destinatario, i PAN ID, e l'header di sicurezza AES-128. In questo caso, rimangono solo 81 byte per il payload. Se sottraiamo 40 byte per l'header IPv6 e 8 byte per l'header UDP, restano soli 33 byte per i dati applicativi. Con la compressione 6LoWPAN, si possono recuperare decine di byte, rendendo il sistema molto più pratico.

Il dispatch byte: il segnale d'ingresso di 6LoWPAN

Ogni frame 6LoWPAN inizia con un **dispatch byte** (o dispatch field), che identifica il tipo di header 6LoWPAN che segue. Questo permette a 6LoWPAN di essere estensibile: nuovi tipi di header possono essere aggiunti in futuro semplicemente definendo nuovi valori del dispatch byte. I valori principali includono: "Not a 6LoWPAN frame" (00xxxxxx), "IPv6 header completamente inline" (01000001, per frame che non possono essere compressi), "LOWPAN_IPHC" (011xxxxx, per header compressi con IPHC), "LOWPAN_MESH" (10xxxxxx, per frame con mesh header), e "LOWPAN_FRAG" (11xxxxxx, per frame frammentati).

Frammentazione: l'adattamento all'MTU

Quando un pacchetto IPv6 supera la dimensione di un frame 802.15.4, il layer 6LoWPAN lo frammenta in pezzi. Ogni frammento include un **Fragment Header** con tre informazioni: il *datagram_size* (dimensione totale del datagramma originale), il *datagram_tag* (un identificatore che accomuna tutti i frammenti dello stesso datagramma) e il *datagram_offset* (posizione di questo frammento nel datagramma originale, in unità da 8 byte). Il ricevente usa queste informazioni per riassemblare il datagramma originale una volta ricevuti tutti i frammenti.

Il primo frammento include anche il *datagram_size* ma non il *datagram_offset*; tutti i frammenti successivi includono entrambi. Un aspetto critico è che la frammentazione introduce latenza e rischio di inefficienze: se un singolo frammento va perso, l'intero datagramma deve essere ritrasmesso. Per questo motivo, la compressione dell'header è fondamentale: più l'header viene compresso, meno spesso è necessario frammentare.

Compressione HC1 e IPHC: ridurre l'overhead IPv6 a pochi byte

La compressione degli header IPv6 in 6LoWPAN sfrutta una serie di osservazioni sul contesto tipico di una rete LoWPAN: gli indirizzi IPv6 link-local possono essere derivati dagli indirizzi MAC (EUI-64), il prefisso di rete è condiviso da tutti i nodi della stessa LoWPAN (shared context), e molti campi dell'header hanno valori comuni (Traffic Class e Flow Label spesso sono 0, la versione è sempre 6).

HC1 è la prima tecnica di compressione, ottimizzata per gli indirizzi link-local. L'idea è: se gli indirizzi IPv6 sorgente e destinazione possono essere interamente derivati dagli indirizzi MAC presenti nel frame 802.15.4, non c'è bisogno di ripeterli nell'header IPv6. Questo può comprimere un header IPv6 da 40 byte a soli 2-3 byte (dispatch byte + HC1 byte + hop limit). Quando si aggiunge la compressione dell'header UDP (HC2), si risparmiano altri 4-6 byte.

IPHC (Improved Header Compression) è la versione migliorata, che funziona anche per indirizzi globali unicast e multicast. IPHC usa il concetto di **shared context**: mittente e ricevente condividono uno o più prefissi di rete comuni (identificati da un Context ID a 4 bit), e gli indirizzi possono essere compressi omettendo il prefisso noto. Con IPHC e indirizzi global e Context 0, l'header IPv6 può essere ridotto a soli 4 byte. Aggiungendo la compressione NHC per UDP, si arriva a un overhead totale di soli 6 byte contro i 48 byte di IPv6+UDP non compressi.

Il Mesh Header e il routing mesh-under

6LoWPAN supporta anche il **routing mesh-under**, ovvero il forwarding di pacchetti multi-hop a livello link (layer 2) piuttosto che a livello IP (layer 3). Quando viene usato il mesh-under, un frame 802.15.4 può essere inoltrato da nodo a nodo usando gli indirizzi MAC, senza che ogni nodo intermedio ricostruisca gli indirizzi IPv6 del pacchetto. Il **Mesh Header** di 6LoWPAN contiene l'indirizzo originale (il vero mittente del pacchetto), il final destination (il vero destinatario) e un campo Hops Left (decrementato ad ogni hop, come il TTL di IP). Ogni router intermediario legge il Mesh Header per determinare dove inoltrare il frame, ma gli indirizzi IP nel header IPv6 rimangono quelli di sorgente e destinazione finali.

> 📖 **APPROFONDIMENTO (LIBRO)**
> Il capitolo 16 del libro di Vasseur e Dunkels fornisce una descrizione dettagliata di tutti i meccanismi 6LoWPAN con esempi numerici del risparmio in byte. Il libro di Shelby e Bormann "6LoWPAN: The Wireless Embedded Internet" è la trattazione di riferimento completa sulla specifica.

🗺️ 9. RPL – Routing Protocol for Low-Power and Lossy Networks

Perché serviva un nuovo protocollo di routing per LLN

I protocolli di routing tradizionali di Internet (come OSPF o RIP) sono stati progettati per reti stabili con link affidabili ad alta banda. Le LLN (Low-power and Lossy Networks) IoT sono fondamentalmente diverse: link instabili con perdita di pacchetti elevata e variabile, nodi con risorse computazionali e memoria limitate, topologie potenzialmente di migliaia di nodi, pattern di traffico dominati dalla comunicazione many-to-one (raccolta dati verso un sink), nodi che dormono periodicamente. Adattare OSPF o RIP a questo contesto sarebbe come cercare di navigare in un labirinto sterrato con un autobus da 50 posti.

Il Working Group ROLL dell'IETF ha sviluppato **RPL** (Routing Protocol for Low-power and Lossy Networks, pronunciato "ripple") come risposta specifica a questi requisiti. RPL è un protocollo di routing proattivo a vettore distanza che costruisce una struttura logica chiamata **DODAG** (Destination-Oriented Directed Acyclic Graph).

Il DODAG: la struttura di routing di RPL

Un DODAG è un grafo orientato aciclico con una singola radice. Ogni nodo nella rete ha un percorso verso la radice (che tipicamente è l'LBR — LoWPAN Border Router, il gateway verso Internet), e il grafo è costruito in modo che non ci siano cicli di routing. La "destinazione" del DODAG è la radice stessa, e per questo si dice *destination-oriented*.

Ogni nodo in un DODAG ha un **Rank**, che rappresenta la sua distanza dalla radice in termini di qualità del percorso (non necessariamente hop count). Il Rank è calcolato usando una metrica definita dall'**Objective Function (OF)** del DODAG. Regola fondamentale: il Rank di un nodo deve essere superiore al Rank del suo parent — questa regola garantisce l'assenza di cicli.

Il DODAG supporta naturalmente la comunicazione **many-to-one (MP2P)** — ogni nodo invia dati verso la radice seguendo i link verso i parent — che è il pattern dominante nelle reti di raccolta dati IoT. Con i messaggi DAO (che vedremo tra poco), RPL costruisce anche il percorso inverso (**one-to-many, P2MP**) dalla radice verso i nodi foglia, necessario per inviare comandi o aggiornamenti. La comunicazione **punto-a-punto (P2P)** avviene percorrendo il DODAG verso l'alto fino a un antenato comune, poi scendendo verso la destinazione.

La metrica ETX: misurare la qualità dei link wireless

Uno degli insight più importanti di RPL è l'uso dell'**ETX (Expected Transmission Count)** come metrica di routing invece del semplice hop count. ETX misura il numero medio di trasmissioni (incluse le ritrasmissioni) necessarie per inviare con successo un pacchetto su un link. Su un link perfetto ETX = 1; su un link con perdita del 50% dei pacchetti ETX = 2; su un link molto lossy ETX può essere molto alto. La metrica ETX cattura in modo molto più preciso la qualità reale di un link wireless rispetto al semplice hop count.

L'ETX viene calcolato empiricamente osservando la percentuale di successo delle trasmissioni. Non è specificato dall'IETF come va calcolato (può essere con link-layer acknowledgment, con trasmissioni di probe, ecc.) — questo è deliberato per permettere flessibilità implementativa. L'ETX è una quantità locale a ogni link e si propaga attraverso i messaggi DIO per permettere ai nodi di calcolare il costo del percorso verso la radice.

L'Objective Function: la mente di RPL

L'**Objective Function (OF)** è il componente di RPL che definisce l'obiettivo dell'ottimizzazione: quale metrica usare, come calcolare il Rank dai valori della metrica, come scegliere il parent migliore. RPL è progettato per supportare multiple OF per lo stesso DODAG, permettendo a diverse applicazioni di ottimizzare per obiettivi diversi. Per esempio:

Una OF può specificare "trova il percorso con il minimo ETX aggregato". Un'altra può specificare "trova il percorso con minima latenza che non attraversa nodi battery-operated". Una terza può cercare "il percorso con il massimo link quality minimo lungo tutto il percorso". RPL permette di costruire molteplici DODAG istances (identificate da un RPLInstanceID) con OF diverse, e i nodi possono partecipare a più istanze contemporaneamente, instradando traffico diverso su DODAG diversi.

I messaggi DIO e DAO: costruire e mantenere il DODAG

RPL usa due tipi principali di messaggi ICMPv6 per costruire e mantenere il DODAG. I messaggi **DIO (DODAG Information Object)** fluiscono "verso il basso" (dalla radice verso le foglie) e contengono informazioni sul DODAG: il DODAGID (identificatore univoco della radice), l'RPLInstanceID, il Rank del nodo che invia il DIO, il DODAGSequenceNumber (versione del DODAG), e i parametri dell'OF. Un nodo che riceve DIO da un vicino può valutare se quel vicino è un buon candidato come parent, usando l'OF per confrontare i costi dei percorsi disponibili.

I messaggi **DAO (Destination Advertisement Object)** fluiscono "verso l'alto" (dalle foglie verso la radice) e portano informazioni sui prefissi raggiungibili verso il basso. Grazie ai DAO, i parent possono costruire tabelle di routing che permettono di instradare traffico dalla radice verso destinazioni specifiche (P2MP). I DAO includono anche il meccanismo del **Reverse Route Stack**: quando un nodo intermedio non può memorizzare lo stato di routing (nodi con poca memoria), aggiunge il proprio indirizzo al RRS, permettendo il source routing in discesa.

Il Trickle Timer: comunicare solo quando necessario

Uno dei problemi di qualsiasi protocollo di routing per reti dense è la generazione di traffico di controllo ridondante: se molte nodi sono in una stessa rete stabile, non c'è bisogno che ognuno invii continuamente DIO. Il **Trickle Timer** risolve questo problema con un meccanismo adattivo elegante.

L'algoritmo Trickle mantiene un periodo I (inizialmente I_min) che raddoppia fino a I_max ogni volta che la rete è stabile. Quando il timer scade, il nodo invia un DIO solo se il numero di DIO "consistenti" sentiti dagli altri nodi (il contatore c) è inferiore a una soglia k. Se c ≥ k, la trasmissione viene soppressa perché ci sono già abbastanza nodi che stanno propagando la stessa informazione. Quando viene rilevata un'inconsistenza nel DODAG (ad esempio, un nodo vede un DIO con informazioni diverse dalle proprie), il timer viene resettato a I_min, causando una rapida diffusione delle nuove informazioni.

Loop avoidance e repair del DODAG

RPL implementa meccanismi specifici per evitare i loop di routing, che in una rete IoT lossy sarebbero particolarmente dannosi. La regola del Rank garantisce che i pacchetti fluiscano sempre "verso l'alto" (verso rank minori) in direzione MP2P, ma è necessario prevenire che un nodo scelga come parent un nodo con Rank uguale o superiore al proprio.

Quando un link fallisce, RPL usa due meccanismi di repair. Il **local repair** cerca rapidamente un nuovo parent tra i vicini conosciuti, senza necessità di ricostruire l'intero DODAG. Il **global repair** è guidato dalla radice che incrementa il DODAGSequenceNumber: questo scatena una ricostruzione completa del DODAG da zero, partendo dalla radice. Il global repair è più lento ma garantisce la convergenza verso uno stato ottimale.

> 🔑 **RPL IN SINTESI: I PUNTI CHIAVE DA RICORDARE**
> **DODAG:** Grafo diretto aciclico con radice (LBR), percorsi da ogni nodo alla radice.
> **Rank:** Indica la distanza relativa dalla radice secondo l'OF, non può diminuire allontanandosi dalla radice.
> **DIO:** Messaggi verso il basso per costruire il DODAG (dal root ai nodi).
> **DAO:** Messaggi verso l'alto per popolare il routing table discendente.
> **ETX:** Metrica chiave che misura la qualità dei link (trasmissioni medie necessarie).
> **Trickle Timer:** Riduce il traffico di controllo nelle reti stabili; si resetta sulle inconsistenze.
> **OF:** Definisce l'obiettivo di ottimizzazione (es. min ETX, min latenza, ecc.).

🌐 10. CoAP – Constrained Application Protocol

Perché non usare HTTP direttamente?

HTTP (HyperText Transfer Protocol) è il protocollo applicativo su cui è costruito il Web, e il suo modello RESTful (GET, POST, PUT, DELETE su risorse identificate da URI) è estremamente intuitivo e potente. Perché non usarlo direttamente nei dispositivi IoT? La risposta è semplice: HTTP è progettato per macchine potenti con connessioni affidabili e ad alta banda. Un singolo header HTTP può facilmente superare i 500 byte, e HTTP richiede TCP come trasporto — con tutto il suo overhead di three-way handshake, sliding window e gestione delle connessioni.

Su una rete IoT con frame da 127 byte e processori da pochi MHz, HTTP è semplicemente troppo pesante. **CoAP** (Constrained Application Protocol, RFC 7252) risolve questo problema: è un protocollo applicativo RESTful ottimizzato per dispositivi vincolati, che usa UDP come trasporto e ha header di soli 4 byte fissi. CoAP è essenzialmente un HTTP leggero specificamente progettato per l'IoT, con meccanismi aggiuntivi per gestire l'inaffidabilità intrinseca di UDP.

I quattro tipi di messaggi CoAP

CoAP definisce quattro tipi di messaggi, identificati da un campo Type (T) a 2 bit nell'header. Un messaggio **CON (Confirmable)** richiede un ACK da parte del ricevente: se l'ACK non arriva entro un timeout, il mittente ritrasmette (exponential backoff). Un messaggio **NON (Non-confirmable)** viene inviato senza aspettare ACK, adatto per dati periodici di bassa importanza (ad esempio letture di temperatura ogni secondo). Un messaggio **ACK** conferma la ricezione di un CON. Un messaggio **RST (Reset)** indica che il ricevente non può elaborare il messaggio (ad esempio, il contesto della transazione è stato perso).

Le richieste CoAP usano metodi analoghi a HTTP: **GET** (recupera la rappresentazione di una risorsa), **POST** (crea o processa una risorsa), **PUT** (aggiorna o crea una risorsa con un URI specifico) e **DELETE** (cancella una risorsa). Le risposte hanno codici di risposta con struttura simile a HTTP: 2.xx per successo, 4.xx per errori client, 5.xx per errori server.

Il meccanismo Observe: subscription push-model

Un'estensione fondamentale di CoAP è il meccanismo **Observe** (RFC 7641). Il modello request-response standard è appropriato per richieste occasionali, ma per monitorare continuamente il valore di una risorsa (ad esempio la temperatura letta ogni secondo da un sensore), un client che interroga ripetutamente il server sprecherebbe molta energia. Observe introduce un modello publish-subscribe: il client invia una GET con l'opzione Observe, e il server lo registra come "osservatore". D'ora in poi, ogni volta che il valore della risorsa cambia significativamente, il server invia automaticamente una notifica al client, senza che questo debba richiederla.

Il client può annullare la subscription inviando una GET senza l'opzione Observe, oppure il server può terminare la notifica se il client non risponde per troppo tempo. Questo meccanismo è molto più efficiente del polling continuo e si adatta naturalmente ai pattern di comunicazione IoT.

Scoperta delle risorse: Link Format e /.well-known/core

Come fa un client IoT a sapere quali risorse sono disponibili su un dispositivo CoAP? La risposta è il meccanismo di **resource discovery** basato sul Link Format (RFC 6690). Ogni dispositivo CoAP espone un URI speciale `/.well-known/core` che, quando interrogato con GET, restituisce una lista delle risorse disponibili nel formato Link Format. Ogni risorsa è descritta da un URI, attributi opzionali (tipo di risorsa, tipo di contenuto, dimensione massima) e relazioni tra risorse. Questo permette la scoperta automatica senza configurazione manuale.

Sicurezza in CoAP: DTLS

Poiché CoAP usa UDP, non può usare TLS (che richiede TCP). La soluzione è **DTLS** (Datagram TLS, RFC 6347), la versione di TLS adattata per protocolli datagram. DTLS fornisce le stesse garanzie di sicurezza di TLS — autenticazione, confidenzialità, integrità — ma gestisce la natura stateless di UDP (ogni pacchetto può arrivare fuori ordine o non arrivare). DTLS aggiunge un overhead significativo (tipicamente 13-29 byte per record), ma è necessario per applicazioni che richiedono sicurezza end-to-end.

CoAP definisce quattro modalità di sicurezza: *NoSec* (nessuna sicurezza, solo per reti chiuse e fidate), *PreSharedKey* (chiave condivisa pre-configurata), *RawPublicKey* (chiavi asimmetriche senza certificato), e *Certificate* (PKI completa con certificati X.509). Per dispositivi molto vincolati, PreSharedKey è la scelta più comune per il suo basso overhead computazionale.

> 🔑 **COAP VS HTTP: CONFRONTO CHIAVE**
>
> | Caratteristica | HTTP | CoAP |
> |---|---|---|
> | Trasporto | TCP | UDP |
> | Header minimo | ~500 byte | 4 byte |
> | Sicurezza | TLS | DTLS |
> | Modello | Request-Response | Request-Response + Observe |
> | Multicast | No | Sì (UDP multicast) |
> | Target | PC, server | Dispositivi IoT vincolati |

💻 11. Sistemi Operativi per IoT

Perché servono OS specializzati per l'IoT

Un sistema operativo general-purpose come Linux o Windows richiede decine di megabyte di RAM e qualche gigabyte di storage. Un microcontrollore IoT tipico ha pochi kByte di RAM e poche decine di kByte di Flash. La necessità di OS specializzati per l'IoT non è solo quantitativa ma qualitativa: un OS IoT deve gestire la comunicazione wireless come funzionalità di prima classe, supportare nativamente i pattern di duty-cycling per il risparmio energetico, e funzionare in modo affidabile con stack protocollari ridotti.

Contiki OS: il sistema delle protothreads

Contiki (2003, SICS — Swedish Institute of Computer Science, oggi Thingsquare) è stato il primo OS per smart objects a garantire la comunicazione IP completa. Il suo contributo più importante è probabilmente lo stack **uIP** (micro-IP): un'implementazione completa di TCP/IP che occupa solo 5 kByte (e uIPv6 solo 11 kByte), rendendo possibile l'uso di IP su microcontrollori con pochissima memoria.

Il modello di programmazione di Contiki si basa sui **protothreads**, una forma di cooperative multitasking estremamente efficiente in termini di memoria. Un protothread è una coroutine leggera che può "dormire" in attesa di un evento (timer scaduto, pacchetto ricevuto) senza occupare uno stack completo come un thread tradizionale. Grazie ai protothreads, un nodo Contiki può eseguire più processi concorrenti con un overhead di memoria minimo.

Un altro elemento cruciale del toolchain Contiki è il simulatore **Cooja**: permette di simulare reti di centinaia di nodi virtuali sul proprio PC, con la stessa base di codice che verrà eseguita sui nodi reali. Questo è fondamentale per lo sviluppo e il debugging di protocolli IoT prima del deployment su hardware reale.

TinyOS: event-driven per le WSN

TinyOS (2000, Berkeley-Stanford) è il sistema operativo più usato in ambito accademico per le wireless sensor network. È scritto in **nesC**, un dialetto del C che introduce il concetto di "componenti" collegati tra loro tramite interfacce. Il modello di esecuzione è puramente *event-driven*: non ci sono thread né loop periodici nel codice applicativo; invece, ogni funzionalità è implementata come gestione di eventi (interrupt, timer, ricezione di pacchetti). Questo modello è naturalmente efficiente perché il processore va in sleep automaticamente quando non ci sono eventi da processare.

TinyOS ha introdotto molte idee fondamentali per l'IoT, incluso il concetto di "split-phase operation": quando un'operazione può richiedere tempo (ad esempio inviare un pacchetto), TinyOS restituisce immediatamente il controllo al chiamante e notifica il completamento tramite un evento separato. Questo evita i blocchi e permette al sistema di rimanere reattivo.

FreeRTOS: il real-time per l'industria

FreeRTOS è l'OS di riferimento per applicazioni IoT industriali che richiedono garanzie di timing real-time. A differenza di Contiki e TinyOS, FreeRTOS usa un modello di programmazione **preemptive e multithreaded**: i task vengono schedulati da un kernel real-time in base a priorità configurabili, e il task a priorità più alta viene sempre eseguito per primo. Questo garantisce latenze di risposta predeterminate, essenziali in applicazioni di controllo industriale.

FreeRTOS è portato su oltre 50 microprocessori/controllori ed è supportato da un ecosistema enorme di driver e librerie. Amazon ha acquisito FreeRTOS nel 2017 e lo ha integrato con AWS IoT, diventando uno standard de-facto per l'IoT industriale. Il suo svantaggio principale per applicazioni battery-powered è il maggiore overhead di memoria rispetto a Contiki o TinyOS.

> ℹ️ **CONFRONTO OS PER IOT**
>
> | OS | Anno | Modello | Linguaggio | Stack IP | Punto di forza |
> |---|---|---|---|---|---|
> | Contiki | 2003 | Protothreads (cooperative) | C | uIP (5kB) | Leggerezza, Cooja simulator |
> | TinyOS | 2000 | Event-driven | nesC | uIP | Efficienza WSN, academia |
> | FreeRTOS | ~2003 | Preemptive multithreaded | C | uIP/lwIP | Real-time garantito, industria |

🔒 12. Sicurezza nell'IoT

Perché la sicurezza IoT è particolarmente difficile

La sicurezza informatica è una sfida ovunque, ma nell'IoT assume dimensioni particolari. I dispositivi IoT spesso non possono permettersi le operazioni crittografiche computazionalmente costose tipiche dei sistemi tradizionali. Hanno memoria insufficiente per certificati X.509 complessi. Sono spesso fisicamente accessibili agli attaccanti (si pensi a un sensore outdoor). Una volta distribuiti, possono rimanere non aggiornabili per anni. E soprattutto, un attacco a una rete IoT non è solo un problema di dati: può avere conseguenze nel mondo fisico reale (disattivare un sistema di monitoraggio medico, manomettere un sistema di controllo industriale).

Le principali categorie di attacchi alle reti IoT

Il corso analizza le principali tipologie di attacchi specifici per le reti IoT, che si differenziano dagli attacchi tradizionali per la loro natura distribuita e fisica. Un **attacco Sinkhole** avviene quando un nodo malicioso annuncia falsamente di avere un percorso eccellente verso la radice (basso ETX, alta qualità), attirando il traffico di routing di altri nodi — che poi vengono "inghiottiti" nel sinkhole. Questo può distorcere l'intera rete e permettere all'attaccante di intercettare o modificare dati.

L'attacco **Wormhole** è particolarmente insidioso: due nodi maliciosi fisicamente lontani nella rete sono collegati da un tunnel privato (il "wormhole"). Uno riceve pacchetti e li trasmette istantaneamente all'altro tramite il tunnel, facendo sembrare i due nodi vicini tra loro. Questo crea percorsi di routing fittizi che non rispettano la topologia reale e possono essere usati per manipolare il routing del protocollo RPL.

Il **Sybil attack** prevede che un singolo nodo malicioso si presenti alla rete con molteplici identità false, amplificando la propria influenza sui meccanismi di routing e sicurezza. L'attacco **Selective Forwarding** è più sottile: un nodo compromesso si comporta correttamente la maggior parte del tempo, ma occasionalmente "dimentica" di forwarding certi pacchetti, degradando le prestazioni senza essere ovviamente malicioso. Gli attacchi **Denial of Service (DoS)** e **Replay** completano il quadro: il primo consiste nel generare traffico eccessivo per esaurire le risorse dei nodi; il secondo nel registrare e ritrasmettere pacchetti legittimi per causare azioni non autorizzate.

Le contromisure: sicurezza a più livelli dello stack

La difesa nelle reti IoT deve essere multi-livello, perché nessun singolo meccanismo è sufficiente. Al **livello MAC (IEEE 802.15.4)**, lo standard supporta cifratura AES-128 e autenticazione dei frame tramite Message Authentication Code (MAC/MIC). Questo protegge la comunicazione tra nodi adiacenti ma non su percorsi multi-hop (ogni hop decifra e ricifra). Al **livello di rete**, IPSec fornisce autenticazione (AH — Authentication Header) e/o cifratura+autenticazione (ESP — Encapsulating Security Payload) end-to-end. RPL stesso supporta messaggi sicuri con cifratura e autenticazione dei DIO e DAO. Al **livello di trasporto**, DTLS protegge le comunicazioni CoAP con autenticazione, integrità e cifratura end-to-end.

Una stratificazione completa protezione-per-livello è lo standard raccomandato per applicazioni critiche, anche se introduce overhead computazionale e di memoria che può essere problematico per i dispositivi più vincolati. La scelta del livello di sicurezza appropriato dipende dall'applicazione: un sensore di temperatura in un ufficio ha requisiti molto diversi da un sensore in un impianto industriale critico o in un pacemaker.

> ⚠️ **SICUREZZA IOT: SFIDE APERTE**
> **Aggiornamenti firmware:** Come aggiornare in modo sicuro il software di migliaia di dispositivi distribuiti? Un meccanismo di OTA (Over-The-Air) update è essenziale ma complicato da implementare sicuramente su reti lossy.
>
> **Gestione delle chiavi:** Come distribuire e aggiornare le chiavi crittografiche su dispositivi con poca memoria e capacità computazionale limitata?
>
> **Autenticazione dei dispositivi:** Come verificare che un nodo che si unisce alla rete IoT sia effettivamente un dispositivo legittimo e non un intruso?

📶 13. LoRa e LoRaWAN

Il contesto: reti LPWAN per IoT geograficamente distribuito

IEEE 802.15.4 è eccellente per reti IoT di piccole-medie dimensioni all'interno di un edificio o di un campus. Ma cosa succede quando si devono connettere sensori distribuiti su aree geografiche estese — una smart city, un'intera area agricola, una rete di contatori intelligenti su scala urbana? La soluzione tradizionale è usare reti cellulari (2G/3G/4G), ma queste comportano costi di abbonamento significativi e consumi energetici incompatibili con batterie di lunga durata.

Le reti **LPWAN (Low-Power Wide Area Network)** sono nate per rispondere a questa esigenza: devono coprire distanze dell'ordine dei chilometri (invece dei decine di metri di 802.15.4), con consumi energetici bassissimi e costi ridotti, accettando come trade-off data rate molto bassi (da qualche byte a qualche centinaio di byte al secondo). Tra le varie tecnologie LPWAN — NB-IoT, Sigfox, LTE-M — **LoRa/LoRaWAN** è quella che il corso analizza in dettaglio, per la sua diffusione e il modello di approccio tecnico interessante.

LoRa: la modulazione CSS che permette il lungo range

LoRa (Long Range) è una tecnologia di modulazione radio proprietaria sviluppata da Semtech. La caratteristica distintiva è l'uso della **CSS (Chirp Spread Spectrum)** modulazione, che rappresenta i dati come chirp — segnali la cui frequenza varia linearmente nel tempo (un chirp "up" ha frequenza crescente, un chirp "down" ha frequenza decrescente). La CSS offre eccellente robustezza contro le interferenze, la multipath propagation e il rumore — caratteristiche essenziali per comunicazioni a lungo raggio in ambienti urbani e rurali.

Il parametro più importante di LoRa è lo **Spreading Factor (SF)**, che può variare da SF7 a SF12. Un SF più alto significa: più simboli per bit inviato (spreading più aggressivo), quindi maggiore robustezza contro il rumore (migliore link budget), ma anche data rate più basso e tempo di trasmissione più lungo (Time on Air). SF7 offre ~5,5 kbps con range di qualche km; SF12 offre solo ~250 bps ma con range fino a 15-20 km in condizioni ottimali. La scelta dello SF è fondamentale per bilanciare range e data rate.

Spreading Factor	Data Rate approssimativo	Range tipico	Sensibilità
SF7	~5.5 kbps	~2 km urbano	-123 dBm
SF9	~1.8 kbps	~5 km	-129 dBm
SF12	~250 bps	~15 km rurale	-137 dBm

LoRaWAN: l'architettura di rete

LoRaWAN (LoRa Wide Area Network) è il protocollo di livello MAC e di rete costruito sopra la modulazione LoRa. Definisce l'architettura della rete e le regole di accesso al canale. L'architettura di LoRaWAN è una **stella di stelle**: gli End Node (dispositivi IoT) comunicano con i Gateway tramite link radio LoRa; i Gateway sono connessi a un Network Server tramite backhaul IP (Ethernet, cellular); il Network Server gestisce la deduplicazione, il routing e il controllo della rete e si interfaccia con gli Application Server che elaborano i dati applicativi.

Un aspetto importante è che in LoRaWAN i gateway sono semplici forwarder: non fanno routing, inviano semplicemente al Network Server tutti i pacchetti che ricevono. Il Network Server si occupa della logica di rete. Questo significa che lo stesso pacchetto può essere ricevuto da più gateway (in overlapping coverage areas) e il Network Server deduplica: una semplificazione architetturale significativa che elimina la necessità di coordinazione tra gateway.

Le tre classi di dispositivi LoRaWAN

LoRaWAN definisce tre classi di dispositivi che bilanciano diversamente tra consumo energetico e latenza di ricezione. I dispositivi **Class A** (tutti i dispositivi LoRaWAN devono supportare almeno questa classe) aprono due brevi "finestre di ricezione" (RX1 e RX2) solo subito dopo ogni trasmissione. Il resto del tempo il ricevitore è spento. Questo è il profilo energetico più efficiente, ma significa che il server può inviare dati a un dispositivo Class A solo immediatamente dopo che il dispositivo ha trasmesso — latenza potenzialmente alta.

I dispositivi **Class B** aggiungono finestre di ricezione pianificate (beacon), permettendo al server di inviare dati in momenti predefiniti, riducendo la latenza a scapito di un maggiore consumo. I dispositivi **Class C** mantengono il ricevitore quasi sempre acceso (tranne durante le trasmissioni), garantendo latenza minima ma con consumo energetico significativamente superiore, quindi tipicamente adatti a dispositivi collegati alla rete elettrica.

Attivazione e sicurezza in LoRaWAN

LoRaWAN usa due chiavi AES-128 per la sicurezza: la **NwkSKey** (Network Session Key) per l'autenticazione a livello di rete e l'integrità dei messaggi (Message Integrity Code), e la **AppSKey** (Application Session Key) per la cifratura end-to-end del payload applicativo. Questa separazione garantisce che il Network Server possa verificare l'autenticità dei messaggi ma non accedere ai dati applicativi cifrati — i dati applicativi sono accessibili solo all'Application Server.

L'attivazione dei dispositivi (ovvero la negoziazione delle chiavi) avviene in due modalità. Con **OTAA (Over-The-Air Activation)**, il dispositivo e il Network Server si autenticano tramite una chiave radice (AppKey) e derivano dinamicamente NwkSKey e AppSKey durante il join procedure — questo è il metodo più sicuro perché le chiavi di sessione cambiano ad ogni join. Con **ABP (Activation By Personalization)**, NwkSKey e AppSKey vengono pre-programmate nel dispositivo in fabbrica — più semplice ma meno sicuro.

Il meccanismo di **ADR (Adaptive Data Rate)** permette al Network Server di ottimizzare lo SF di ogni dispositivo in base alla qualità del link osservata: dispositivi vicini ai gateway vengono impostati su SF più bassi (data rate più alto, tempo on air inferiore) mentre quelli lontani usano SF più alti. Questo ottimizza sia il throughput della rete sia la durata della batteria dei dispositivi.

> 📖 **LORA VS IEEE 802.15.4: QUANDO USARE QUALE?**
> **IEEE 802.15.4 + 6LoWPAN + RPL:** Reti mesh all'interno di edifici o campus, dove i nodi fungono anche da router e si necessita di bassa latenza e IP nativo.
>
> **LoRaWAN:** Sensori sparsi su aree geografiche estese (città, campi agricoli), dove la comunicazione è prevalentemente uplink (sensore → server), le trasmissioni sono rare (pochi pacchetti al giorno) e la durata della batteria si misura in anni.

> ✅ **STACK PROTOCOLLARE IOT COMPLETO**
> Di seguito lo stack protocollare completo del corso, dal livello fisico all'applicazione:

CoAP (Applicazione)
UDP / DTLS (Trasporto)
IPv6 + RPL (Rete)
6LoWPAN (Adattamento)
IEEE 802.15.4 MAC + Energy-efficient protocols
IEEE 802.15.4 PHY (Fisico)

Stack IoT su IEEE 802.15.4

Applicazione LoRaWAN
LoRaWAN MAC (Class A/B/C)
Network Server
LoRa (CSS modulation)
Sub-GHz PHY

Stack LoRaWAN